CS-3510-B Exam 1

Elijah Martin McCauley

TOTAL POINTS

97 / 110

QUESTION 1

Problem 1 18 pts

1.1 (i) **3 / 3**

✓ **- 0 pts** *Correct (False)*

- 3 pts Incorrect

1.2 (ii) **3 / 3**

✓ **- 0 pts** *Correct (False)*

- 3 pts Incorrect

1.3 (iii) **3 / 3**

✓ **- 0 pts** *Correct (False)*

- 3 pts Incorrect

1.4 (iv) **3 / 3**

✓ **- 0 pts** *Correct (False)*

- 3 pts Incorrect

1.5 (v) **3 / 3**

✓ **- 0 pts** *Correct (True)*

- 3 pts Incorrect

1.6 (vi) **3 / 3**

✓ **- 0 pts** *Correct (True)*

- 3 pts Incorrect

QUESTION 2

Problem 2 20 pts

2.1 (i) **4 / 4**

✓ **- 0 pts** *Correct (4)*

- 2 pts Correct, but no work shown

- 2 pts Incorrect, but efforts made in the right direction

- 4 pts Missing answer

2.2 (ii) **4 / 4**

✓ **- 0 pts** *Correct (10)*

- 2 pts Correct, but no work shown

- 2 pts Incorrect, but efforts made in the right direction

- 4 pts Incorrect and no work/Missing answer

2.3 (iii) **4 / 4**

✓ **- 0 pts** *Correct (27)*

- 2 pts Correct, but no work shown

- 2 pts Incorrect, but efforts made in the right direction

- 4 pts Incorrect and no work/Missing answer

2.4 (iv) **4 / 4**

✓ **- 0 pts** *Correct (1)*

- 2 pts Correct, but no work shown

- 2 pts Incorrect, but efforts made in the right direction

- 4 pts Incorrect and no work/Missing answer

2.5 (v) **4 / 4**

✓ **- 0 pts** *Correct (5)*

- 2 pts Correct, but no work shown

- 2 pts Incorrect, but efforts made in the right direction

- 4 pts Missing answer

QUESTION 3

Problem 3 22 pts

3.1 (i) **11 / 17**

+ 17 pts Correct

+ 5 pts Base Case

+ 5 pts Correctly using CountFrequency() on two half lists

+ 2 pts Correctly checking for max from left and right.

+ 5 pts Splitting list into two and making recursive calls

✓ **+ 11 pts** *Inefficient Solution*

+ 6 pts Used a sorting algorithm

+ 6 pts Did not use a divide and conquer algorithm

+ 0 pts Incorrect/Missing answer

1 this costs $$ \mathcal{O}(n) $$ and is ran in the worst case $$ n/2 $$ times for a runtime of $$ \mathcal{O}(n^2) $$

2 there is a way to exploit pairing for an $$ \mathcal{O}(n) $$ solution:

remove two elements from the front of the list. if they're equal, add one of them to the back of the list. if they're not equal, discard both. repeat this process until there's only one element left, at which point it must be the majority.

if you don't like the need for a double-ended queue, maintain two stacks in rounds or turn it into divide and conquer by making a new list and recuring on this list.

3.2 (ii) **5 / 5**

✓ **+ 5 pts** *Correct*

+ 3 pts Partial Credit (like a minor mistake in the recurrence, not specific about which Master's Theorem case used etc.)

+ 0 pts Incorrect/Missing answer

QUESTION 4

Problem 4 20 pts

4.1 (i) **10 / 10**

✓ **- 0 pts** *Correct*

T(n) = 8T(n/8)+O(1); a=8, b=8, d=0

- 1 pts 1 written instead of O(1)

- 3 pts Minor Error

- 6 pts Major Error

- 10 pts No answer/Missing answer

4.2 (ii) **10 / 10**

✓ **- 0 pts** *Correct*

- 3 pts Major error/ Compounded error from part A

- 10 pts Incorrect/Missing answer

QUESTION 5

Problem 5 20 pts

5.1 (i) **15 / 15**

✓ **+ 15 pts** *Correct*

+ 14 pts Ranges swapped

+ 5 pts Base Case

+ 2 pts Using Binary Search as a black-box

+ 5 pts Splitting list into two and making recursive calls on either of them

+ 10 pts Inefficient Solution/Solution with Minor Errors

+ 0 pts Incorrect/Missing answer

5.2 (ii) **5 / 5**

✓ **- 0 pts** *Correct*

- 3 pts No work shown/Major Error

- 5 pts Incorrect/Missing answer

- 1 pts Minor Error in work

QUESTION 6

Problem 6 10 pts

6.1 (i) **3 / 3**

✓ **- 0 pts** *Correct $$(2^n)$$*

- 2 pts $$2n$$

- 3 pts Incorrect/Missing answer

6.2 (ii) **0 / 7**

- 0 pts Correct (52)

- 2 pts Minor FLT error

- 3 pts Major FLT error

- 3 pts 27 is the answer you get if you said $$2n$$ in Part A

- 4 pts No work shown for $$n$$

- 5 pts Shows some logic, but incorrect answer

✓ **- 7 pts** *Incorrect/Missing answer*

GTID: 903684653
NAME: Elijah McCauley

Georgia Institute of Technology

Spring 2023

CS 3510B – Design & Analysis of Algorithms

February 2, 2023

TIME ALLOWED: 75 MINS

Name: Elijah McCauley

GTID: 903684653

GT Username: emccauley6

INSTRUCTIONS TO CANDIDATES

1. Please write your NAME and GTID clearly on all the pages.
2. This examination paper contains **SIX (6)** questions and comprises **ELEVEN (11)** printed pages.
3. ONLY write on the front sheets of paper that are numbered. The backs will not be scanned.
4. Calculators are **NOT** allowed.

I am in aware of and the accordance with Academic Honor Code of Georgia Tech and the Georgia Tech Code of Conduct. I'll use no external help on this test. Also, I have read all the instructions on this page.

Signature: [signature]

GTID: 903684653
NAME: Elijah McGaley

Problem 1 *(18 points; 3 points each)*

Indicate whether the following statements are ***true*** or ***false***.

$n^{\log_2 16} = n^4 \neq n^5$

(i) $16^{\log_2 n} = \Theta(n^5)$

false

(ii) $n^3 = \mathcal{O}(n^{\log_3 13})$

false $n^3 > n^{\log_3 13}$

(iii) $n^{n^3} = \Omega(n^{2^n})$

false n^{2^n} grows faster than n^{n^3}

(iv) $n! = \mathcal{O}(1024^n)$

false $n! > x^n$

(v) $n \log_2 n = \mathcal{O}(n \log_2 n^{1024})$

$n \log_2 n = 1024\, n \log_2 n$

true

(vi) $\log_2 n^3 = \Omega(2^{103})$

$3 \log_2 n > 2^{103}$ true

GTID: 903684653
NAME: Elijah McCauley

Problem 2 *(20 points; 4 points each)*

Compute the following. Your answer should not contain p or q.

(i) 2^{128} (*mod* 127) where 127 is prime.

$2^{126} 2^2 \equiv 2^2 \bmod 127 = \boxed{4}$

$4 \bmod 127$

(ii) 2^{345} (*mod* 11) where 11 is prime.

$(2^{10})^{34} 2^5 \equiv 2^5 \bmod 11$

$32 \bmod 11$

$\boxed{10}$

$2^1 = 2$
$2^2 = 4$
$2^3 = 8$
$2^4 = 16$
$2^5 = 32$

(iii) 3^{3p} (*mod* p) where p is a prime such that $p > 3^3$ and doesn't divide 3^3.

3^{P-1} $\quad a = 3$

$P = P \qquad 3^P \equiv 3 \bmod p$

$3^P 3^P 3^P \bmod p$

$3^P \equiv 3 \bmod p$ so $3^P 3^P 3^P \equiv 3 \cdot 3 \cdot 3 \bmod p = \boxed{27}$

GTID: 907684653
NAME: ElijahMcCauley

(iv) $(2^q - 1)^2 \pmod q$ where q is prime and doesn't divide a^q where $q > 10$ and $1 < a < 10$.

$(2^{a^q} - 1)(2^2 - 1)$ $2^{q^2} - 2^2 + 1 \bmod q$

$a = 2$ $4 - 4 + 1$ $1 \bmod q = 1$

$2^{q^2} \bmod q$ $\boxed{1}$ $\#2 \cdot 2^q \bmod q$

$2^{q-1} 2^{q-1} 2^2 \bmod q$ $\#2 \cdot 2^{2-1} \cdot 2 \equiv 4 \bmod q = 4$

(v) Find an integer x such that $x^{103} \equiv 4 \pmod{11}$ where $1 \leq x \leq 5$.

$1^{103} \not\equiv 4 \bmod 11$

$(2^{10})^{10} 2^3 \equiv 2^3 \bmod 11 = 8$ so not 2

$(3^{10})^{10} 3^3 \equiv 3^3 \bmod 11 \equiv 27 \bmod 11 \equiv 5 \bmod 11 \not\equiv 4 \bmod 11$

$(4^{10})^{10} 4^3 \equiv 4^3 \bmod 11 \equiv 64 \bmod 11 \equiv 9 \bmod 11$

$(5^{10})^{10} 5^3 \equiv 5^3 \bmod 11 \equiv 125 \bmod 11 \equiv 4 \bmod 11$

$\boxed{X = 5}$

GTID: 903484653
NAME: ElijahMcCaylee

Problem 3 *(22 points)*

You are given an unsorted array S of length n. An element is known as *majority element* if it appears in the list more than $n/2$ times. You can check if two items are equal in $\mathcal{O}(1)$ time, but you cannot check their ordering. In other words, you cannot determine if $A[i] < A[j]$ or $A[i] > A[j]$, so you cannot sort items in the array or find their medians. Perhaps they are image files, and are incomparable.

You are given a black-box algorithm called $CountFrequency(A, l, r, value)$. It takes in four parameters: the unsorted array A, the left bound l, the right bound r, and the *value* that you are looking for. In $O(n)$ time, it outputs a frequency of how many element=*value* are present in between the bounds, inclusive l and r.

For example, the unsorted array $[2, 12, 2, 2, 2, 4, 2]$ has the majority element output of 2, as 2 appears more than $\lfloor 7/2 \rfloor = 3$ times in the array.

(i) (17 points) Provide a Divide & Conquer algorithm by either writing the pseudo-code **OR** describing it in english to find the majority element in array S in strictly $\mathcal{O}(n \log n)$ time.

No additional data structures are allowed!

call countfreq log n times

2(n/2) n

To find the majority element we must proceed by splitting the array into 2 parts: left and right. we call the countFrequency black box algorithm function on the left half using the value found at A[0]. if it returns a value greater than $\lfloor n/2 \rfloor$ then we return the number at A[0] that we checked. If it does not return a high enough value then we know it is not the majority value. We do the same on the right side of the array.

If there is an odd number, then the left subarray will be larger.

We divide until we have reached the point where we have n singular elements. We then begin the recombine process by comparing each number to its neighbor in O(1) time. If a number is the same as its neighbor, then we run the countFrequency blackbox algorithm on that number in the original array →

GTID: 103684657
NAME: Elijah McCarley

If it returns a value $> \lfloor n/2 \rfloor$ then we have found the majority value. If not, we keep going with our comparisons until it happens again and we repeat this process until we have found the majority value.

Because it appears $> n/2$ times, we know that one of the pairs at the end of the split must contain 2 of the majority element, so those are the only values we need to check in the blackbox algorithm. 1

2

(ii) (5 points) Use the Master Theorem to prove the algorithm's time complexity.

$T(n) = 2\left(\frac{n}{2}\right) + O(n)$

$a = 2$
$b = 2$
$d = 1$

$\log_2 2 = 1$

$n^d \quad d = 1$

$\log_2 2 = d = 1$

so

$n^d \log n$

$= O(n \log n)$

GTID: 903684653
NAME: Elijah McClay

Problem 4 *(20 points; 10 points each)*

Answer the following problems analyzing the function. Let n be the power of 8. Suppose we have the following function:

```
def func(n):
    if n == 0:
        return
    for i in [1, 2, 3, 4]:
        func(n/8)
        func(n/8)

    print("Hello World!")
```

n=8

(i) Let $T(n)$ be the number of times "Hello World!" is printed. Find the recurrence relation for $T(n)$.

$T(n) = 8T(\frac{n}{8}) + O(1)$

$\log_8 8 = 1$

$d = 0$

$\log_b a > d$

so

$O(n^{\log_b a})$

$O(n^1)$

(ii) What is the tight upper bound for $T(n)$?

The tight upper bound is $O(n^1)$

GTID: 903684653
NAME: Elijah McCauley

Problem 5 *(20 points)*

Given a sorted array of distinct integers $A[1, ..., n]$, you want to find out whether there is an index i for which $A[i] = i$. If such an index exists, return the index i. Otherwise, return -1.

(i) (15 points) Design a Divide & Conquer algorithm by either writing the pseudo-code **OR** describing it in english that runs in $\mathcal{O}(\log n)$ time.

So what we want to do is essentially call binary search. We set a variable L to = 1 and R to = n. We then find the middle index of the array by $M = \frac{L+R}{2}$. We compare the value A[M] to M. If A[M] = M we return M. If A[M] > M we set R = M-1 and repeat the previous steps. ~~If A[M] < M then we return -1 because the array is sorted, if~~ The value of A[M] can never be less than M because the array is sorted distinct integers so at index 3 for example you can't have A[3] = 2 because there are not 2 numbers < 2 to fit into A[1] and A[2]. The only time A[M] = M is when A[i] = i for all i < M. So when you check the middle value and it's equal to M you stop and return, otherwise you set R = M+1, and repeat finding $M = \frac{L+R}{2}$ and checking A[M] = M.

The algorithm stops when you check A[1] = 1 where you will either return 1 if true and -1 if A[1] ≠ 1.

GTID: 903684653
NAME: Elijah Mccrate

(ii) (5 points) Use the Master Theorem to prove the algorithm's run-time.

$a = 1$ recurse 1 time $T(n) = 1\left(\frac{n}{2}\right) + O(1)$ $d = 0$

$b = 2$ 2 subarrays

$d = 0$ constant time

$\log_b a = \log_2 1$

$a = b^d$

$1 = 2^0$

so

big $O = O(n^d \log n)$

$= O(n^0 \log n)$

$= O(\log n)$

GTID: 907684653
NAME: Elijah Mc Gee

Problem 6 *(Extra Credit; 10 points)*

You have a deck of 52 cards numbered 1 through 52, where the top card in number 1, the 2nd from the top is number 2, and so on. Let's consider a shuffle of this deck, called a *bridge*. You take the top 26 cards of the deck and the bottom 26 cards of the deck and you perfectly interleave them, adjusting their positions as follows:

Card 1 → 2nd from top
Card 2 → 4th from top
Card 3 → 6th from top
...
Card 26 → bottom card
Card 27 → top card
Card 28 → 3rd from top
...

Here's another way to describe a bridge:

Card $j \rightarrow$ position $2j \pmod{53}$

(*Note*: We are starting at position 1, with no card at position 0. That's we used $(mod\ 53)$ and not $(mod\ 52)$).

(i) *(3 points)* Consider what happens when you perform this *bridge* n times. Fill in the blank where j is the original position of the card and k is the position after n bridges.

$$k \equiv \underline{\quad 2^n \quad} j \pmod{53}$$

27
1
28
2

(ii) *(7 points)* Find some $n \neq 0$ such that if you perform a bridge n times all the cards will be in their original positions. Justify your answer.
Hint: Use your answer from part A.

END OF EXAM